CS2009 - Design and Analysis of Algorithm
Fall 2026
Project Description
Total Marks: 100. You need to work as a Group of 3

In this project, you are required to

1) [40 Points] Implement the following divide and conquer algorithms using any language of your choice with user defined input size of at least 10. Your programs must also run on input sizes in the millions (at least 1,000,000). You do not need to show any GUI.

General requirements (apply to all parts):

- **Language:** Any language may be used. C++, Java, C# or Rust are recommended for inputs in the millions. Python is allowed, but take care with its recursion limit and with copying lists inside recursive calls.
- **Input size:** Every program must run correctly for $n = 10$ and must complete for $n = 1{,}000{,}000$. The input size is chosen by the user at run time and must not be hard-coded.
- **Library routines:** Do not call built-in sorting, big-integer multiplication or closest-pair routines inside your algorithms. You may use them only to check that your answers are correct.
- **Random data:** All random inputs must be produced by a generator with a fixed, recorded seed so that the same data can be generated again.
- **Large values:** Use 64-bit integers wherever a count can exceed about 2×10^9 (for example, the number of inversions).

a) [Source: Introduction to the Design and Analysis of Algorithms, Anany Levitin, 3rd Edition, Exercise 5.1 (2), Page 174]

1. Write pseudocode for a divide-and-conquer algorithm for finding values of both the largest and smallest elements in an array of n numbers.
2. Set up and solve (for $n = 2^k$) a recurrence relation for the number of key comparisons made by your algorithm.
3. How does this algorithm compare with the brute-force algorithm for this problem?

b) [Source: Introduction to the Design and Analysis of Algorithms, Anany Levitin, 3rd Edition, Exercise 5.1 (3), Page 174]

1. Write pseudocode for a divide-and-conquer algorithm for the exponentiation problem of computing a^n where n is a positive integer.
2. Set up and solve a recurrence relation for the number of multiplications made by this algorithm.
3. How does this algorithm compare with the brute-force algorithm for this problem?

c) [Source: Introduction to the Design and Analysis of Algorithms, Anany Levitin, 3rd Edition, Exercise 5.1 (9), Page 175]

1. Let $A[0..n-1]$ be an array of n real numbers. A pair $(A[i], A[j])$ is said to be an ***inversion*** if these numbers are out of order, i.e., $i < j$ but $A[i] > A[j]$. Design an $O(n \log n)$ algorithm for counting the number of inversions.

d) [Source: Introduction to the Design and Analysis of Algorithms, Anany Levitin, 3rd Edition, Section 5.2, Page 176]

1. What is the worst-case and best-case complexity of the quicksort algorithm?

e) [Source: Introduction to the Design and Analysis of Algorithms, Anany Levitin, 3rd Edition, Exercise 5.5 (1), Page 197]

1. For the one-dimensional version of the closest-pair problem, i.e., for the problem of finding two closest numbers among a given set of n real numbers, design an algorithm that is

directly based on the divide-and-conquer technique and determine its efficiency class.

2. Is it a good algorithm for this problem?

f) [Source: Algorithm Design, Jon Kleinberg, 1st Edition, Page 242]

1. Solved Exercise 1

g) [Source: Algorithm Design, Jon Kleinberg, 1st Edition, Page 244]

1. Solved Exercise 2

h) [Source: Algorithm Design, Jon Kleinberg, 1st Edition, Exercises (1-7), Page 246-249]

1. Any three exercises

2) [10 Points] Implement the following divide and conquer algorithms using any language of your choice. Generate 10 random sample inputs for each of the two problems below, with the least input size > 100 and the largest input size in the millions (at least 1,000,000), and save them on text files. So in summary you have 10×2 different files of varying complexities: 10 files for closest pair of points and 10 files for integer multiplication.

(i) Finding the closest pair of points

(ii) Integer Multiplication

What you have to do:

- **Closest pair of points (2-D):** implement the $O(n \log n)$ divide and conquer algorithm (Kleinberg, Section 5.4). Output the closest pair and the distance between them.
- **Integer multiplication:** implement Karatsuba's divide and conquer algorithm, switching to ordinary (schoolbook) multiplication when the numbers become small. Output the product.

Input files for closest pair of points

File	Number of points	Content
cp01–cp04	10^3, 10^4, 10^5, 10^6	Uniform random real coordinates in $[0, 10^6]$
cp05	1,000,000	Clustered: points around 10 random centers
cp06	1,000,000	Integer coordinates in a small range, so many duplicate points (closest distance 0)
cp07	100,000	All points on one vertical line
cp08	100,000	Equally spaced grid (many tied closest pairs)
cp09	1,000,000	Closest pair placed across the middle dividing line
cp10	101	Small file for showing the algorithm step by step

Input files for integer multiplication (each file holds two non-negative integers)

File	Digits per number	Content
im01–im04	10^3, 10^4, 10^5, 10^6	Random digits, no leading zero
im05	1,000,000 and 10	Numbers of very different lengths
im06	100,000	One number is all 9s
im07	99,999	Odd length, with many zeros
im08	1,000,000	Both numbers are powers of 10
im09	1 and 1,000,000	One number is 0
im10	101	Small file for showing the algorithm step by step

File format: plain text. Closest pair: the first line is n, followed by one "x y" pair per line. Integer multiplication: two lines, each holding one number in decimal digits.

What to submit: your generator program, the seed used for each file, and only the files smaller than 1 MB. The large files will be regenerated from your generator during marking. Groups using Python may limit integer multiplication to 100,000 digits if they cannot finish the 1,000,000-digit files, but must state this in the report.

3) **[5 Points]** Apply these algorithms on the input datasets.
4) **[30 Points]** You need to show a very nice user interface where user can select any input file (assumed valid), and then show the working of both algorithms using nice user interface. For small inputs, show the algorithm step by step; for large inputs (in the millions), show the final result and the running time.
5) **[15 Points]** You are required to submit strictly a 3-page report (maximum 18% similarity) written in LaTeX. Submit the LaTeX source files (.tex, .bib and figures) together with the PDF report. The report must have the following sections:
    a. Abstract
    b. Introduction
    c. Your proposed system (make a clear diagram of your system here along with discussion)
    d. Experimental Setup (The details of each input dataset including input size, range and method used to generate the random numbers)
    e. Results and Discussion (Here, show results from benchmarks i.e. correct solution obtained using various files)
    f. Conclusion
    g. References

Submission Instruction:
The project should be submitted in three phases. You have to submit each deliverable within the due date. In case of non-compliance the maximum obtainable marks would be reduced by 25% each day after the due date (cumulative: 75% after one day, 50% after two days, 25% after three days, and 0 after four days).

Submission 1:
- Deliverables: Project part 1.
- Due Date: October 22, 2026

Submission 2:
- Deliverables: Project part 2, 3, and 4.
- Due Date: November 10, 2026

Submission 3:
- Deliverables: Project part 5 (LaTeX source files and PDF report).
- Due Date: November 20, 2026